paper-writing-agent

Evidence-grounded, anti-AI-slop manuscript writing for CS & software papers

Write papers that read as precise human work, and pass review.

Objective: maximize acceptance at high-impact venues.

A hybrid: a deterministic Python core plus a Claude Code plugin.

The tool lints its own prose with its own rules.

pip install paper-writing-agent

github.com/taichi8912/paper-writing-agent

The problem: AI-assisted writing leaves fingerprints

Reviewers penalize the tells. Four recurring failure modes:

1. AI vocabulary and structure

delve, underscore, pivotal, em dashes, the rule of three. Read as machine-written.

2. Over-claimed statistics

Bare P-values; "significant" with no test and no explicit verdict.

3. Hallucinated citations

Invented keys; references that do not say what the sentence claims.

4. Untraceable and inconsistent

Undefined terms, forward references, numbers with no source.

Objective and quality hierarchy

Objective
Maximize the probability of acceptance at a high-impact venue.

When two goals conflict, the earlier one wins:

1. Accuracy	every claim is factual and backed by data
2. Clarity	an expert understands it on the first read
3. Conciseness	no redundant words or expressions
4. Professionalism	an appropriate academic register

Accuracy is never traded for brevity; clarity is never traded for polish.

Eight design pillars

1 Evidence grounding

Every claim resolves to a source.

2 Anti-AI-slop

Remove fingerprints; precise register.

3 Statistical honesty

Significance needs a test and a verdict.

4 Single source of truth

One owner per rule and per state.

5 Plan, confirm, execute

Verify after. The user decides.

6 Reproducibility

Re-runnable stats; auditable changes.

7 Configurable, not rigid

Venue, style, strictness are settings.

8 Dogfooding

The project lints its own prose.

Technical core I: the anti-AI-slop linter

Deterministic detection with a suggested fix for every finding.

- **Three vocabulary tiers**
  forbidden, replace-with-alternative, filler phrases
- **Structural patterns**
  copula avoidance, participle padding, negative parallelism, the rule of three
- **Em dashes: zero tolerance by default**
- **Masking**
  code, math, and citations are never flagged; offsets are preserved for exact locations

Before

We delve into a pivotal, novel method, leveraging a robust pipeline to enhance throughput.

↓

After

We study a new method that uses a reliable pipeline to increase throughput.

pwa lint <paths>

errors fail the run; warnings advise; --format json for tooling; the rules ship in the linter.

Technical core II: honesty and grounding

Statistical honesty

- Every P-value needs an explicit verdict
- Name a statistical test once, at first use
- Notation follows the house style

Accepted

significant ($P = 2.6 \times 10^{-5}$; Welch's t-test)

Flagged

"... higher than B (P = 0.01)." (no verdict)

pwa stats <paths>

Citation grounding

- BibTeX becomes a searchable context index
- JSON-Schema anti-hallucination gates
- Coverage, no placeholders, no slop in prose
- Ground keys in real prior usage (verified)

Never invent a key

If none fits: leave % TODO: cite -- <claim>

pwa bib build / validate / ground

Architecture: a hybrid you can verify

The plugin orchestrates; the core decides. The rules are unit-tested.

Dogfooding: the repository lints its own prose with its own linter, in pre-commit and CI.

Configurable, not rigid

An interactive wizard and three presets. Sensible defaults; everything overridable.

pwa init strict-high-IF | balanced | lenient

You configure

- Target field and venue tier
- Citation style (numeric / author-year)
- Spelling (US / UK)
- P-value notation and significant figures
- Section structure and figure references
- Hedging strictness
- Operating language (en / zh / ja)
- Project-local forbidden-word extensions

Always on

- Significance needs a test and a verdict
- Citations are never fabricated
- No speculative edits to unread files
- The user holds decision authority
- Accuracy is never demoted

Friendly out of the box; never a straitjacket.

Get started

```
$ pip install paper-writing-agent
 $ pwa init my-paper/
  $ pwa check my-paper/
     errors block submission; warnings advise; install the Claude Code plugin to drive it agentically.
```

Write papers that read as precise human work, and pass review.

Open source, MIT. Generic and domain-agnostic: no private or project-specific material.

github.com/taichi8912/paper-writing-agent